

Production ML Project Setup

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University

Let me share some common software development practices



WHAT IT COSTS YOU

- No record of **why** anything changed
- Two people cannot touch the same file
- The version that worked is the one you overwrote
- "It ran yesterday" is not a recoverable state

TODAY

The habits that replace all of this — the shell, git, repo hygiene, secrets and Docker.

Motivation

Several failures you might hit while working on your projects.

✗ WORKS ON MY MACHINE

Your script runs. Your classmate's identical checkout does not. Nothing says which versions it needs.

A LEAKED KEY

You push `.env` with a live API key. Public repositories are scanned automatically, and the bill is yours.

AN UNREPRODUCIBLE RESULT

A number in your report that nobody — including you, three weeks later — can regenerate.

What this lecture is

IT IS

- The habits that prevent those failures
- Shell, git, repo hygiene, environments, secrets, Docker
- Things you will use in **every** later lecture

IT IS NOT

- A tools tour for its own sake
- Lockfiles and multi-environment workflows — out of scope
- CI/CD or `docker compose` — out of scope

Everything here is POSIX-common: it works in **zsh** (the macOS default) and in bash.

The Shell

Where everything else happens

Navigating

```
pwd          # where am I?  
ls           # what is here?  
ls -la       # ...including dotfiles, with permissions and sizes  
cd projects/agents # go somewhere  
cd ..        # up one level  
cd ~         # home
```

PATHS

Absolute starts at `/` · **relative** starts where you are. `.` is here, `..` is up, `~` is home.

USE TAB

Tab completes names. It is not a shortcut — it is how you avoid typos in paths you cannot see.

`ls -la` **matters**: `.env` and `.gitignore` are invisible to plain `ls`.

Files and directories

```
mkdir -p data/raw      # -p creates parents as needed
touch notes.md         # create an empty file
cp config.yaml backup/ # copy
cp -r data/ backup/    # ...a directory needs -r
mv old.py new.py       # move = rename
rm scratch.txt         # delete
```

READ A FILE

`cat` dumps it · `less` pages it (`q` quits) · `head` /
`tail` show the ends

RESPECT RM -RF

Recursive, forced, **no trash and no undo**. Type the path, read it, then press enter.

Inspecting and combining

The shell's real power is not the commands — it is joining them.

```
grep "api_key" config.py      # find text in a file
find . -name "*.py"          # find files by name
wc -l data.csv                # count lines
history | grep git | tail -5  # the last 5 git commands you ran
```

PIPE

`a | b` — send a's output into b

REDIRECT

`>` writes to a file (overwrites) · `>>` appends

Permissions and execution

```
chmod +x run.sh      # make it executable
which python3        # which one actually runs?
echo $PATH           # the list of places the shell looks
```

A script needs two things to run as `./run.sh`:

- 1 A **shebang** on the first line — `#!/usr/bin/env bash` — so the system knows what interprets it
- 2 The **execute bit** — `chmod +x run.sh` — so the system is allowed to

Without them you get *permission denied* — the most common first-week error.

Environment variables

```
export OPENAI_API_KEY="sk-..." # set for this shell session
echo $OPENAI_API_KEY             # read it back
env | grep API                    # list what is set
```

WHY THIS MATTERS

Configuration lives **outside** your code. The same script runs against different keys, endpoints and settings without editing a line.

WHERE THIS GOES NEXT

`export` lasts one session. A `.env` file makes it repeatable — and is the reason the secrets block exists later today.

Running things

```
python3 train.py --epochs 10    # arguments  
echo $?                          # exit code of the last command: 0 = success
```

0 means success. Anything else is failure.

It is how every script, pipeline and CI system decides whether your program worked — and **Ctrl-C** is how you stop one that never will.

Git

Not a backup tool — a record of decisions

The three trees

Do not memorise commands. Learn **where a file is**, and every command becomes obvious.



git status tells you where every file sits between these three

every git command is a move between two of these

The everyday loop

```
git clone <url>           # get a repo
git status                 # where is everything? (read this constantly)
git add file.py           # working tree → index
git commit -m "message"   # index → history
git push                  # your history → the remote
git pull                  # the remote → you
```



THE ONE HABIT THAT MATTERS

Run `git status` before and after every command until it is boring. It tells you which of the three trees your file is in.

Reading status

```
$ echo "hello" > a.txt
$ git status --short
?? a.txt          # untracked – git does not know about it

$ git add a.txt
$ git status --short
A  a.txt          # staged – it will be in the next commit

$ git commit -m "first"
$ git log --oneline
9b52242 first
```

Two characters tell you everything: **??** untracked · **A** added · **M** modified · **D** deleted.

Branching

```
git checkout -b feature/retrieval  # create and switch  
git checkout main                 # switch back  
git branch                        # list
```

WHY BRANCH

A branch is a cheap, named line of work. Break it and you throw the branch away, not your project.

IN THIS COURSE

One branch per experiment. Your `main` should always be a state that runs.

A conflict, on purpose

Two branches changed the same line. Git will not guess.

```
$ git merge feature  
CONFLICT (content): Merge conflict in a.txt  
Automatic merge failed; fix conflicts and then commit the result.
```

```
<<<<<< HEAD  
from main  
=====  
from feature  
>>>>>> feature
```

HEAD is **your** side. Below the **=====** is theirs.

Resolving it

WHAT YOU DO

- Open the file
- Delete the markers
- Leave the text you want
- `git add` the file
- `git commit`

WHAT NOT TO DO

- Do not keep both sides "just in case"
- Do not delete the file to make it stop
- Do not `--force` anything to escape

A conflict is git refusing to silently lose someone's work. It is a feature.

The remote and the pull request

```
git push -u origin feature/retrieval # publish your branch
```

WHY NOT PUSH TO MAIN

- `main` stays a state that runs
- A PR is a place to *read* a change before it lands
- Review catches what tests do not

THE FLOW

- Branch → commit → push
- Open a pull request
- Discuss, amend, push again
- Merge, then delete the branch

You will use this in every group project, and in Lecture 11 when an agent writes the diff.

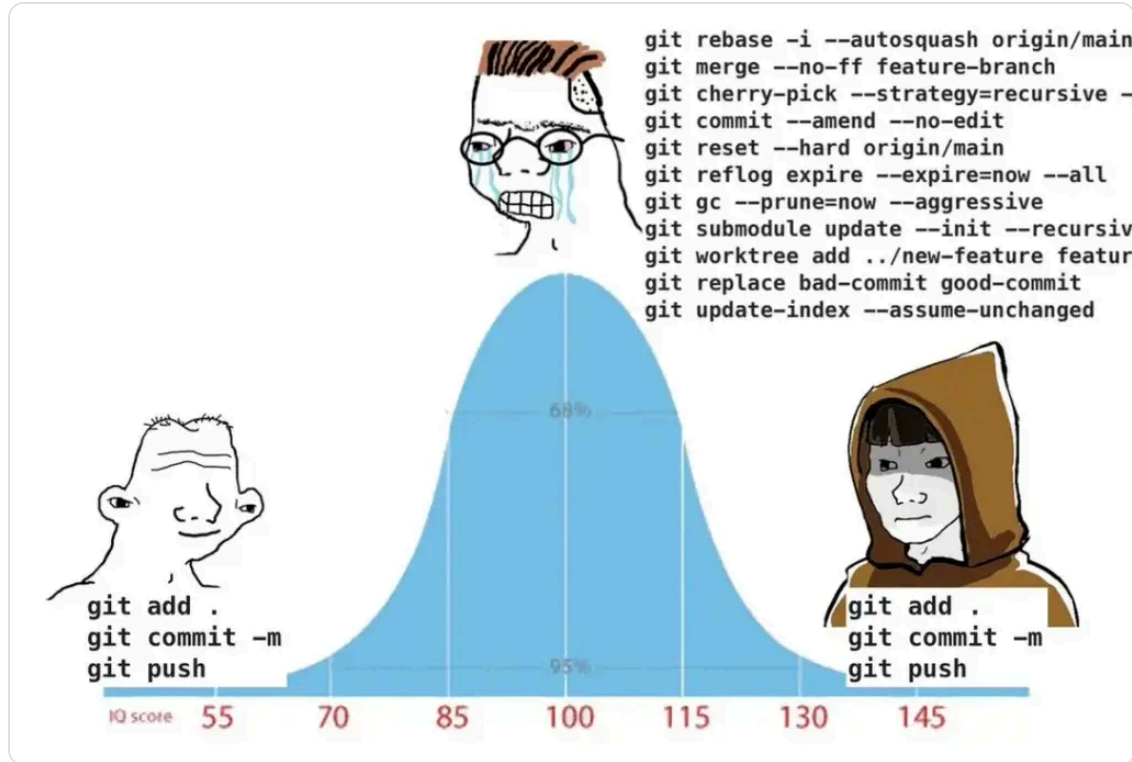
DISCUSSION



Is this a good habit?

How often should you actually **commit** — and how often should you **push**? What is the cost of getting each one wrong in a four-person project?

Commit small, push often



COMMIT

- One coherent change per commit — a commit you cannot describe in one line is two commits
- Commits are free. Losing an afternoon is not

PUSH

- At least daily. Unpushed work exists on exactly one laptop
- A branch nobody has seen for a week is a merge conflict growing quietly

The loop really is three commands. The middle of that curve is where people lose time.

Repo Files & Secrets

What a project must carry

README

The first file anyone opens, including you in six months.

MUST ANSWER

- What is this?
- How do I run it?
- What do I need first?

GOOD ENOUGH

Ten honest lines beat a beautiful document that is out of date. Update it in the same commit that changes behaviour.

.gitignore

Files git should never track: secrets, caches, data, virtual environments.

```
.env  
.venv/  
__pycache__/  
*.pyc  
data/  
.DS_Store
```

THE TRAP THAT CATCHES EVERYONE

`.gitignore` only affects files git is **not yet tracking**. Adding a file to it does nothing if you already committed that file.

Untracking something you already committed

```
echo ".env" >> .gitignore  
git rm --cached .env      # stop tracking; keep your local copy  
git commit -m "stop tracking .env"
```

WHAT THIS DOES

Removes the file from the index so future commits ignore it. Your file stays on disk.

WHAT IT DOES NOT DO

It does **not** remove the file from past commits. If it held a secret, that secret is still in history.

requirements.txt and pinning

```
langgraph==0.2.14      ← pinned: exactly this version  
langchain-openai       ← unpinned: whatever is newest today
```

WHY PIN

An unpinned dependency installs a different version next week. The same repo then behaves differently for you and for everyone else.

WHERE IT BITES

Later lectures use fast-moving APIs where an unpinned import genuinely breaks between one week and the next.

WHAT TO PIN

Anything you import directly, and anything whose API moves fast. In this course that means **every agent framework you use.**

HOW MUCH IS ENOUGH

`package==1.2.3` on the things you named. Pinning the entire transitive tree is a lockfile's job, and out of scope here.

Your environment

A `requirements.txt` is a list. An **environment** is where that list gets installed — one per project, never your system Python.

VENV + PIP

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip freeze > requirements.txt
```

UV — SAME MODEL, FASTER

```
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
uv add langgraph==0.2.14
```

ACTIVATION IS PER SHELL

Every new terminal needs `source` `.venv/bin/activate` again. If `which python3` does not point inside `.venv/`, you are not in it.

NEVER COMMIT IT

`.venv/` is a build artefact, not source. It goes in `.gitignore` — you rebuild it from `requirements.txt`.

.env

```
# .env – never committed  
OPENAI_API_KEY=sk-proj-...  
ANTHROPIC_API_KEY=sk-ant-...
```

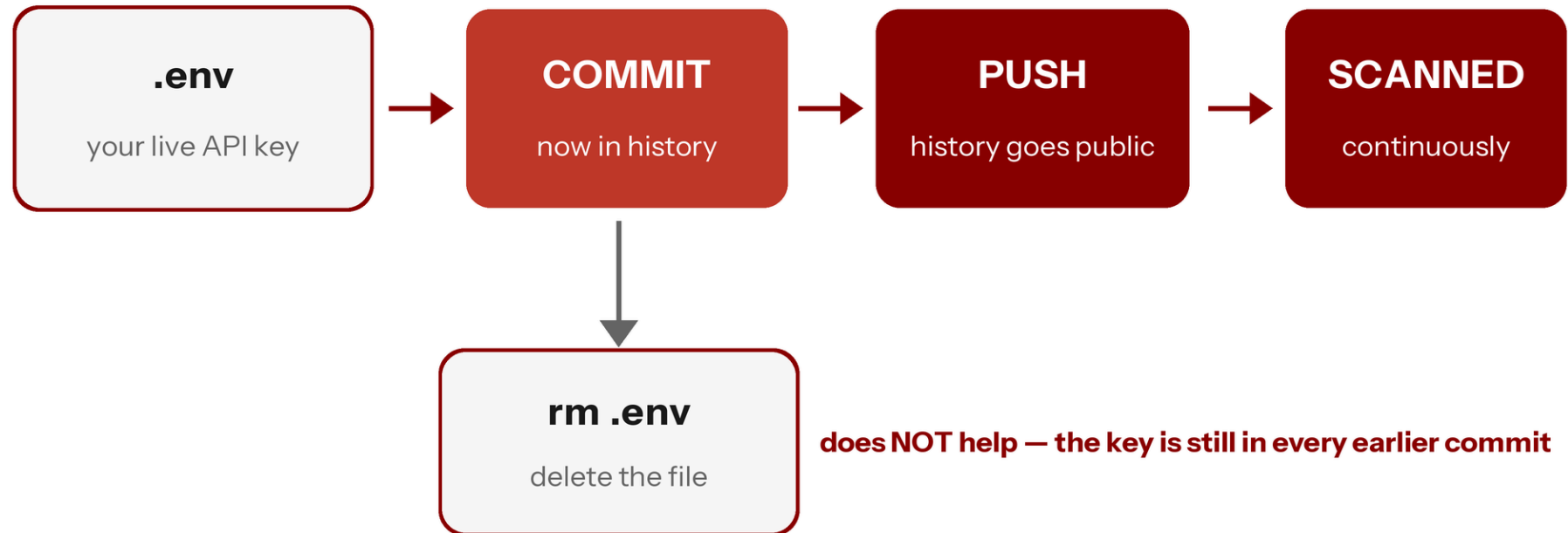
```
# .env.example – always committed  
OPENAI_API_KEY=  
ANTHROPIC_API_KEY=
```

THE PAIR

`.env` holds the real values and is gitignored. `.env.example` holds the **names** with empty values, so a new person knows what to provide.

What actually happens when a key leaks

You will be tempted to delete the file and move on. It does not work.



Do this instead: rotate the key first · then clean history · assume it was used

rewriting history does not un-leak a key that was already public

If it happens to you

1 • ROTATE FIRST

Revoke the key at the provider **before** anything else. Everything else is cleanup; this is the only step that stops the bleeding.

2 • THEN CLEAN HISTORY

Rewrite history to remove the value, and assume it was already copied.

ASSUME COMPROMISE

Public pushes are scanned continuously. Treat any key that was public for any length of time as already used by someone else.

DISCUSSION

Your code runs. Your teammate's identical checkout crashes on import.

Same repository, same commit, same `requirements.txt`, both on Python 3.12. Name three things that could still differ between the two machines.

Three things that could differ

1 • NO ISOLATED ENVIRONMENT

They installed into system Python, where four other projects already live. Something is shadowing something.

2 • UNPINNED VERSIONS

`langchain-openai` with no `==` resolves to whatever shipped that morning.

3 • EVERYTHING BELOW PYTHON

A different OS, a missing system library, a C extension built for another architecture.

You have just fixed the first two. The third is what the rest of this lecture is for.

Docker

"Works on my machine" — solved

Image vs container

IMAGE

A frozen filesystem plus a command. Built once, shared, never changes. Think of it as a class.

CONTAINER

A running instance of an image. Start it, stop it, throw it away, start another. Think of it as an object.

The point: the image carries the OS, the Python version and every dependency — so the thing that ran for you runs for everyone.

A Dockerfile, line by line

```
FROM python:3.12-slim          # the base image
WORKDIR /app                   # where commands run inside the container
COPY requirements.txt .        # dependencies first...
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                       # ...source second
CMD ["python", "app.py"]       # what runs when the container starts
```

Each line becomes a **layer**. Layer order is not style — it decides your build time.

Layer order

Docker caches each layer. Change one, and every layer **after** it is rebuilt.

GOOD — deps before source

FROM python:3.12-slim
cached

COPY requirements.txt
cached

RUN pip install
cached — the slow one

COPY ..
rebuilt: your code changed

CMD [...]
rebuilt

BAD — source before deps

FROM python:3.12-slim
cached

COPY ..
rebuilt: your code changed

COPY requirements.txt
rebuilt

RUN pip install
rebuilt — every single time

CMD [...]
rebuilt

a changed layer invalidates every layer after it

Build and run

```
docker build -t my-project .          # build; -t names it
docker run --env-file .env my-project
```

--ENV-FILE

Passes your `.env` in at run time. The key stays out of the image — an image is shared, so a baked-in key is a leaked key.

.DOCKRIGNORE

Same idea as `.gitignore`, for the build. Keep `.git`, `.env` and `data/` out of the build context.

Summary

COVERED

- Shell: navigate, combine, permissions, env vars
- Git: the three trees, the loop, branches, conflicts
- README · `.gitignore` · pinning · environments · `.env`
- Docker: images, layers, build, run

KEEP THESE

- Run `git status` until it is boring
- Commit small, push often
- A secret in history stays in history
- Rotate first, clean up second
- Dependencies before source, always

References

- Chacon & Straub · **Pro Git**, 2nd ed. · git-scm.com/book — the three trees, §7.7
- Git documentation · **gitignore** · git-scm.com/docs/gitignore
- Docker documentation · **Building best practices** · docs.docker.com
- GitHub Docs · **About secret scanning; Removing sensitive data from a repository**
- Apple Support · **HT208050** — zsh as the default macOS shell

What's Next • Lecture 02: LLM Fundamentals

WHAT'S COVERED

- Tokenization and why tokens are the unit of everything
- Embeddings as a representation
- The transformer, at the mechanism level
- Generation parameters and determinism

HOW TODAY CONNECTS

- You now have a repo, a shell and a key that is not in git
- Every later lecture assumes this setup
- The habits are the point — the tools are how they are expressed

Thank You

Questions & Discussion

Lecture 01 · LLM Agent Development